

MASTER IMPLEMENTATION SPECIFICATION

Multi-Vendor Marketplace + Multi-Tenant Franchise E-Commerce Platform

Platform: Laravel
Architecture: Modular Monolith + Multi-Tenancy
Document purpose: Implementation brief for Claude / engineering team

Business Layer	Purpose
Main Store	Brand-owned direct e-commerce.
Marketplace	Independent vendors sell through the platform; platform earns commission.
Franchise	Official branch stores use the full storefront with branch-specific pricing, inventory and configuration.

1. HIGH-LEVEL ARCHITECTURE

Use a Modular Monolith architecture in Laravel.

```
LARAVEL CORE
|
+-- MAIN STORE
|
+-- MARKETPLACE
|   +-- Independent Vendors
|
+-- FRANCHISE
    +-- Branch Stores
        +-- Tehran
        +-- Shiraz
        +-- Mashhad
```

Shared Core Services:

- Authentication
- Customer Accounts
- Product Catalog
- Orders
- Payments
- Shipping
- Notifications
- Promotions
- Media
- Search
- Reporting Infrastructure
- Admin Infrastructure

Marketplace and Franchise must remain separate business domains. Do NOT mix Vendor and Franchise concepts.

A Vendor is an independent seller. A Franchise is an official branch/store operating under our brand.

2. MAIN STORE

The main website is the brand's primary e-commerce store.

```
vikyplus.ir
```

It should contain standard e-commerce functionality:

- Homepage
- Categories
- Product listings

- Product pages
- Search
- Filters
- Cart
- Checkout
- Customer account
- Orders
- Payments
- Shipping
- Discounts
- Promotions
- Wishlist if available
- Content pages
- Customer support features

The Main Store should also act as the central foundation for the Franchise storefront experience.

3. MULTI-VENDOR MARKETPLACE

Create a complete Multi-Vendor Marketplace system. Independent sellers should be able to register and create their own vendor profiles.

Each vendor should have a dedicated management dashboard.

- Vendor registration
- Vendor approval/rejection
- Vendor profile
- Vendor status
- Vendor staff accounts if required
- Product management
- Price management
- Inventory management
- Order management
- Sales reports
- Financial reports
- Commission visibility
- Settlement history
- Return/refund visibility
- Notifications

The platform administrator must be able to approve, suspend or deactivate vendors.

4. MARKETPLACE PRODUCT MODEL

Design the product architecture carefully. Prefer a canonical/shared product catalog where appropriate.

```
Product
- id
- name
- slug
- description
- category
- brand
- specifications
- media
- variants
- attributes

VendorOffer
- id
- vendor_id
- product_id
- price
- stock
- discount
- status
- SKU
- shipping settings
- fulfillment settings
```

This architecture should allow multiple vendors to sell the same canonical product without unnecessarily duplicating the complete product record.

It should remain flexible enough to support vendor-created products that require administrator approval.

5. MULTI-VENDOR ORDERS

A customer must be able to purchase products from multiple vendors.

IMPORTANT: Vendor ownership must exist at ORDER ITEM level.

Do NOT rely only on vendor_id at the order level.

```
Order
|
+-- Order Item -> Vendor A
+-- Order Item -> Vendor A
+-- Order Item -> Vendor B
```

Each order item must retain enough commercial information to reconstruct the transaction later:

- seller/vendor

- product
- variant
- quantity
- unit price
- discount
- tax if applicable
- commission
- seller payable
- fulfillment status
- refund status

Historical order data must not break if the current product price changes later.

6. MARKETPLACE COMMISSION SYSTEM

The platform earns commission from marketplace transactions. Commission architecture should support:

- Global default commission
- Vendor-specific commission
- Category-specific commission
- Product-specific commission if required
- Percentage commission
- Fixed commission if required
- Future extensibility for hybrid commission rules

Example:

Product Sale: 3,000,000

Platform Commission: 15%

Platform Revenue: 450,000

Vendor Payable: 2,550,000

Commission calculations must be recorded as financial transaction data and remain auditable.

7. VENDOR FINANCIAL LEDGER

Do NOT implement vendor finances as only one editable wallet balance. Use an auditable financial ledger.

- Sale
- Commission
- Refund
- Adjustment
- Settlement
- Charge
- Bonus

- Reversal

Customer Payment

|

v

Order Items

|

+--> Gross Sale

+--> Platform Commission

+--> Vendor Payable

+--> Refund / Adjustment

|

v

Financial Ledger

|

v

Vendor Settlement

Vendor payable balance should be derived from valid financial records.

8. VENDOR SETTLEMENT SYSTEM

Create settlement functionality. The system should track:

- Pending balance
- Available balance
- Held balance if required
- Settled amount
- Refund adjustments
- Settlement requests
- Settlement status
- Settlement date
- Settlement reference
- Administrator notes

Possible settlement states:

- pending
- approved
- processing
- paid
- rejected
- cancelled

The architecture should support manual settlement initially and automated settlement later if needed.

9. MARKETPLACE FULFILLMENT

Marketplace fulfillment must be designed separately from Franchise fulfillment.

- Vendors ship orders themselves
- Platform fulfillment
- Mixed fulfillment models

Each Order Item should retain fulfillment ownership. Returns, cancellations and refunds must work correctly in multi-vendor orders.

10. FRANCHISE / BRANCH NETWORK

Implement a separate Franchise domain. Franchises are official stores/branches of our brand.

- Physical
- Online
- Physical + Online

Every franchise should receive a FULL version of our main e-commerce storefront.

- Do NOT create a simplified Mini Store.
- Do NOT create separate Laravel applications.
- Do NOT duplicate the storefront codebase.

There must be ONE Laravel Core, ONE shared storefront, and MULTIPLE franchise tenants.

11. FRANCHISE STOREFRONT MODEL

Main Store:
vikyplus.ir

Franchise Stores:
tehran.vikyplus.ir
shiraz.vikyplus.ir
mashhad.vikyplus.ir
tabriz.vikyplus.ir

When a customer visits shiraz.vikyplus.ir, Laravel must resolve Shiraz as the active Franchise Tenant.

The customer should see the complete storefront experience, while the commercial data corresponds to the Shiraz branch:

- Shiraz products
- Shiraz availability
- Shiraz prices
- Shiraz inventory
- Shiraz discounts

- Shiraz promotions
- Shiraz banners
- Shiraz contact information
- Shiraz address
- Shiraz shipping methods
- Shiraz delivery settings
- Shiraz business hours
- Shiraz orders
- Shiraz storefront configuration

12. SHARED STOREFRONT CODEBASE

All franchise stores must use the SAME storefront codebase.

```

SHARED STOREFRONT
|
Active Tenant Context
|
+-----+-----+
|   |   |   |
Tehran Shiraz Mashhad

```

A feature developed once should become available to all branches, subject to configuration and feature permissions.

- Wishlist
- New checkout
- Loyalty program
- Auction system
- Step-down sale system
- AI assistant

13. FRANCHISE PRODUCT ARCHITECTURE

Do NOT duplicate complete product information for every branch. Use a shared canonical Product Catalog.

```

Product
- id
- name
- slug
- description
- images
- category
- brand
- specifications
- attributes
- variants

```


BranchOffer / FranchiseProduct

- id
- branch_id
- product_id
- price
- stock
- discount
- is_active
- local SKU if required
- local settings
- fulfillment settings

Example: Jordan 1

Central Store:

Price: 3,071,000

Stock: 24

Shiraz:

Price: 3,150,000

Stock: 4

Tabriz:

Price: 3,090,000

Stock: 7

The product identity and content remain centralized while commercial information can differ by branch.

14. PRODUCT VARIANTS AND INVENTORY

Inventory should work at the correct variant level. For footwear, stock must not exist only at product level.

Jordan 1

Color: Chicago

Branch: Shiraz

Size 37 -> 2

Size 38 -> 4

Size 39 -> 1

Size 40 -> 0

Size 41 -> 3

Design inventory tables so stock can be managed per:

- Branch
- Product
- Variant
- Size
- Color
- SKU

15. FRANCHISE PRICE CONTROL

Each franchise may have its own prices, but central administration should control whether branches are allowed to override prices.

- Central price only
- Branch may set its own price
- Minimum allowed price
- Maximum allowed price
- Maximum discount
- Centrally locked promotion

Do not hard-code one pricing policy into the architecture.

16. FRANCHISE INVENTORY

```
Central Warehouse:
Jordan 1 Size 38 -> 20

Shiraz:
Jordan 1 Size 38 -> 4

Tehran:
Jordan 1 Size 38 -> 7

Mashhad:
Jordan 1 Size 38 -> 2
```

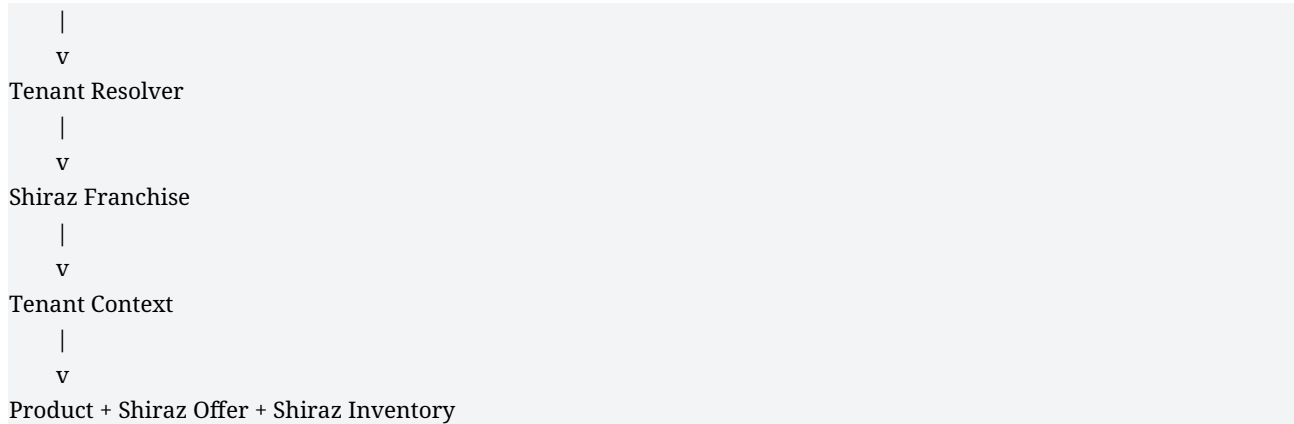
The architecture should remain extensible for future hybrid fulfillment.

```
Order
|
v
Franchise
|
+--> Local Inventory Available
|   |
|   v
|   Local Fulfillment
|
+--> Local Inventory Unavailable
|
v
Central Warehouse
```

17. TENANT RESOLUTION

Use subdomain-based tenant resolution.

```
Request:
shiraz.vikyplus.ir/products/jordan-1
```



The active tenant context should be available consistently throughout the request.

18. TENANT ISOLATION

This is CRITICAL.

Franchise data isolation must exist at backend level. A Shiraz franchise administrator must NEVER access Tehran data by changing URLs, API IDs, manual API requests, frontend requests, or guessed identifiers.

Tenant isolation should be enforced through:

- Middleware
- Policies
- Query scopes
- Service-layer validation
- Repository rules if repositories are used
- Authorization
- Automated tests

Do NOT rely only on frontend restrictions.

19. FRANCHISE ADMINISTRATION

Each franchise should have its own management panel. Depending on permissions, franchise administrators should be able to manage:

- Local inventory
- Local pricing
- Local discounts
- Product activation
- Orders
- Shipping settings
- Store information
- Address

- Phone
- Business hours
- Local banners
- Local promotions
- Reports
- Staff

They must NOT be able to manage another franchise.

20. CENTRAL ADMIN

The central administration system must have complete control.

- Create franchise
- Edit franchise
- Activate/deactivate franchise
- Assign franchise owners
- Assign franchise managers
- Manage franchise permissions
- Manage global products
- Assign products to franchises
- View all franchise inventory
- View franchise prices
- View franchise orders
- View franchise sales
- View franchise performance
- Control branch pricing permissions
- Control feature availability
- Control promotion permissions
- Configure global policies

Central administration should manage Marketplace vendors separately. Do not mix Vendor Management and Franchise Management interfaces unnecessarily.

21. CUSTOMER AUTHENTICATION

Customer authentication should be shared across the ecosystem.

- Main Store
- Franchise Stores
- Marketplace

A customer registered on vikyplus.ir should be able to use the same customer account on shiraz.vikyplus.ir. Tenant/store commercial context must remain separate.

22. CART ARCHITECTURE

Cart must be Store/Branch scoped.

Do NOT accidentally allow one cart to mix Shiraz franchise inventory with Tehran franchise inventory unless cross-branch ordering is explicitly introduced later.

Recommended behavior:

- Customer enters Shiraz store.
- Cart belongs to Shiraz.
- Products use Shiraz price, stock, discount and availability.
- Checkout creates an order associated with Shiraz.
- Store branch_id/store_id on the order.

Marketplace multi-vendor carts are a separate concept and must not be confused with Franchise cross-branch carts.

23. ORDER ARCHITECTURE

Orders must preserve their original commercial context.

- Customer
- Store / Branch
- Seller where applicable
- Order Items
- Original price
- Discount
- Quantity
- Product variant
- Shipping method
- Payment
- Fulfillment ownership
- Commission where Marketplace applies
- Refund information

Do not rely on current product tables to reconstruct historical order pricing.

24. ROLE AND PERMISSION ARCHITECTURE

- Super Admin
- Admin
- Marketplace Manager
- Vendor
- Vendor Staff
- Franchise Owner

- Franchise Manager
- Franchise Staff
- Customer

Authorization must consider Role + Permission + Resource Ownership + Active Tenant where applicable.

25. RECOMMENDED DOMAIN STRUCTURE

```
app/
Domains/
Identity/
Catalog/
Customers/
Orders/
Payments/
Shipping/
Promotions/
Marketplace/
Vendors/
VendorOffers/
Commissions/
Settlements/
MarketplaceFulfillment/
Franchise/
Tenancy/
Branches/
BranchOffers/
BranchInventory/
FranchiseOrders/
FranchiseSettings/
Reporting/
Shared/
Infrastructure/
```

This is conceptual. Adapt it to the existing Laravel project conventions where necessary. Do not blindly restructure working code if the current organization is already clean.

26. DATABASE DESIGN PRINCIPLE

For the current stage, prefer Shared Database + Tenant IDs rather than one database per franchise.

```
branches
branch_users
branch_domains
branch_offers
branch_inventory
branch_settings
orders -> branch_id where applicable
```

Use proper foreign keys and indexes. The architecture must support efficient queries as the number of branches, products, orders and inventory records grows.

27. DO NOT DUPLICATE DATA UNNECESSARILY

Avoid:

30 franchises x 10,000 products = 300,000 duplicated full product records

Prefer:

Canonical Product

+ Branch Offer

+ Branch Inventory

Marketplace:

Canonical Product

+ Vendor Offer

28. FEATURE CONFIGURATION

Support feature flags/settings per franchise.

Shiraz:

Wishlist = ON

Step-down Sale = ON

Local Delivery = ON

Tabriz:

Wishlist = ON

Step-down Sale = OFF

Local Delivery = OFF

29. AUDITING

Important administrative and financial changes should be auditable.

- Vendor commission changed
- Branch price changed
- Inventory manually adjusted
- Settlement approved
- Settlement paid
- Franchise permission changed
- Vendor suspended
- Order manually modified

Store actor, action, affected resource, previous value where appropriate, new value where appropriate, and timestamp.

30. SECURITY

Pay particular attention to:

- Tenant isolation
- Vendor isolation
- Authorization
- IDOR prevention
- Price manipulation
- Cart manipulation
- Inventory race conditions
- Overselling
- Payment verification
- Settlement integrity
- Admin actions
- Mass assignment
- API authorization
- File uploads
- Rate limiting where appropriate

Never trust price, commission, branch_id, vendor_id or payable amounts sent from the frontend. Commercial calculations must be performed and verified server-side.

31. INVENTORY CONSISTENCY

Prevent overselling. Use an appropriate stock reservation / transaction strategy during checkout.

- Concurrent purchases
- Payment pending
- Payment failure
- Order cancellation
- Reservation expiration
- Refund
- Return
- Manual inventory adjustment

32. SEARCH

Search results must respect context.

On shiraz.vikyplus.ir, search should return products available for Shiraz according to configured business rules. Marketplace and Main Store search should follow their own availability rules.

33. SEO AND URLS

```
shiraz.vikyplus.ir/products/jordan-1  
shiraz.vikyplus.ir/category/jordan
```

Design SEO metadata so franchise pages can be customized where appropriate. Consider canonical URLs carefully to avoid unnecessary duplicate-content problems.

34. CUSTOM DOMAINS

Subdomains are the initial strategy. Keep tenant domain architecture extensible enough to support franchise-specific custom domains in the future.

Use a domain mapping model rather than hard-coding franchise names into business logic.

35. REPORTING

- Main Store Sales
- Marketplace GMV
- Marketplace Commission Revenue
- Vendor Sales
- Vendor Payables
- Vendor Settlements
- Franchise Sales
- Sales by Branch
- Sales by Province
- Branch Inventory
- Branch Performance
- Top Products
- Returns
- Refunds
- Discount Performance
- Fulfillment Performance

Not every report must be built immediately, but transactional data should support them later.

36. SCALABILITY PRINCIPLE

Do NOT introduce Microservices just because the system has multiple domains. Start with a clean Modular Monolith.

Keep boundaries clear enough that high-load components can be extracted later if needed.

- Search
- Notifications
- Media processing
- Reporting

- Payments
- Inventory
- Marketplace settlements

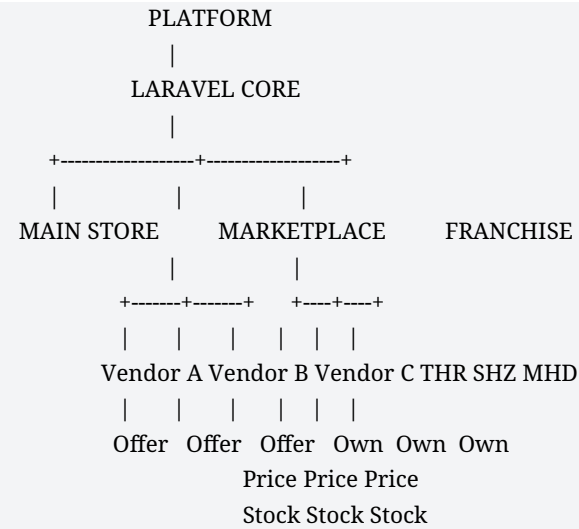
37. IMPORTANT BUSINESS DISTINCTION

Never treat MAIN STORE, MARKETPLACE VENDOR and FRANCHISE BRANCH as identical concepts.

They may share Products, Customers, Payments and Infrastructure, but their business rules are different.

- Marketplace:
 - Independent seller
 - Platform commission
 - Vendor payable
 - Vendor settlement
- Franchise:
 - Official brand branch
 - Branch-specific inventory
 - Branch-specific pricing
 - Branch-specific storefront
 - Central franchise governance
- Main Store:
 - Brand's central direct sales operation

38. TARGET ARCHITECTURE



Shared:

- Authentication
- Canonical Catalog
- Customer Accounts

- Payment Infrastructure
- Order Infrastructure
- Notifications
- Media
- Search Infrastructure
- Reporting Infrastructure

Marketplace-specific:

- Vendors
- Vendor Offers
- Commission
- Vendor Ledger
- Settlement
- Vendor Fulfillment

Franchise-specific:

- Tenant Resolution
- Branches
- Branch Offers
- Branch Inventory
- Branch Storefront Configuration
- Franchise Administration
- Branch Fulfillment

39. IMPLEMENTATION APPROACH

- Inspect the existing Laravel project.
- Understand the current Authentication, Product model, Product variants, Inventory, Cart, Orders, Payments, Admin panel, Frontend architecture, Routes, APIs and Database schema.
- Identify reusable components.
- Do NOT unnecessarily rewrite working functionality.
- Design Marketplace and Franchise domains around the existing stable core where technically reasonable.
- Identify schema changes.
- Identify required migrations.
- Identify security risks.
- Identify potential breaking changes.
- Create an implementation plan.

40. FIRST RESPONSE REQUIRED FROM CLAUDE / ENGINEERING AGENT

Do NOT immediately start making large changes.

First inspect the existing codebase and return a technical implementation plan containing:

- A. Current architecture assessment
- B. Existing components that can be reused
- C. Required new Marketplace components
- D. Required new Franchise components
- E. Database schema changes
- F. Models and relationships
- G. Tenant resolution strategy
- H. Marketplace commission and settlement architecture
- I. Inventory architecture
- J. Cart and order changes
- K. Authentication / authorization changes
- L. Admin panel changes
- M. Required routes / APIs
- N. Security considerations
- O. Migration risks
- P. Testing strategy
- Q. Step-by-step implementation phases

After presenting this plan, wait for approval before performing large destructive or architectural changes.

FINAL TARGET

ONE Laravel Core + ONE shared storefront codebase + ONE shared canonical product catalog where appropriate + MAIN STORE + MULTI-VENDOR MARKETPLACE + MULTI-TENANT FRANCHISE STORES, with strict separation of Marketplace and Franchise business rules.